

CACHING COMPUTATION RESULTS ACROSS MULTIPLE MACHINES

May 16, 2026 | Markus Müller | IBG-3

OVERVIEW

Matching pairs of parentheses (...(..)..) and the Chomsky hierarchy

Incompleteness, Turing machines and λ calculus

Practical Problem description

Functional Programming, random numbers and IO

MUNDANE QUESTIONS, DEEP ANSWERS

Question: Is it possible to create a regular expression to find a **matching** paranthesis in text?

E.g. in source code replace all occurrences of call of f_I by f_{II} :

$$f_I(a, b, c) \rightarrow f_{II}(a, b, c)$$

but also include nested calls like

$$f_I(g(x, y), b, c) \rightarrow f_{II}(g(x, y), b, c)$$

MUNDANE QUESTIONS, DEEP ANSWERS

Question: Is it possible to create a regular expression to find a **matching** paranthesis in text?

E.g. in source code replace all occurrences of call of f_I by f_{II} :

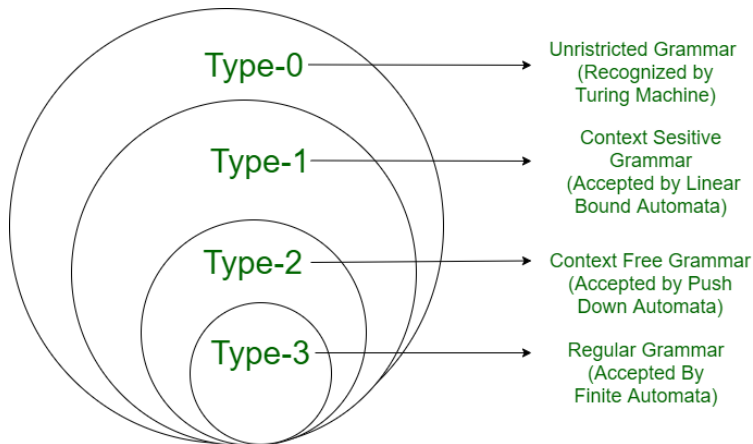
$$f_I(a, b, c) \rightarrow f_{II}(a, b, c)$$

but also include nested calls like

$$f_I(g(x, y), b, c) \rightarrow f_{II}(g(x, y), b, c)$$

Answer: **No!**

CHOMSKY HIERARCHY



regular expressions are Type 3 whereas matching parenthesis require at least Type 2.

CHURCH TURING THESIS/CONJECTURE

Alonzo Church (PhD. adviser of Alan Turing): "No computational procedure will be considered as an algorithm unless it can be represented as a Turing Machine"



EQUIVALENCE OF TURING MACHINES

Fun facts:

EQUIVALENCE OF TURING MACHINES

Fun facts:

- All modern programming languages are Turing complete (can simulate a Turing machine).

EQUIVALENCE OF TURING MACHINES

Fun facts:

- All modern programming languages are Turing complete (can simulate a Turing machine).
- Even minimal languages (e.g. only a while loop) are Turing complete.

EQUIVALENCE OF TURING MACHINES

Fun facts:

- All modern programming languages are Turing complete (can simulate a Turing machine).
- Even minimal languages (e.g. only a while loop) are Turing complete.
- In particular the extremely minimal λ calculus is. (proven not only according to the Church Turing Conjecture)
e.g. in python: `lambda x: x**2`

EQUIVALENCE OF TURING MACHINES

Fun facts:

- All modern programming languages are Turing complete (can simulate a Turing machine).
- Even minimal languages (e.g. only a while loop) are Turing complete.
- In particular the extremely minimal λ calculus is. (proven not only according to the Church Turing Conjecture)
e.g. in python: `lambda x: x**2`
- As a corollary “functional languages” (which have no variables but only expressions) like Haskell, jax, or julia are.

EQUIVALENCE OF TURING MACHINES

Fun facts:

- All modern programming languages are Turing complete (can simulate a Turing machine).
- Even minimal languages (e.g. only a while loop) are Turing complete.
- In particular the extremely minimal λ calculus is. (proven not only according to the Church Turing Conjecture)
e.g. in python: `lambda x: x**2`
- As a corollary “functional languages” (which have no variables but only expressions) like Haskell, jax, or julia are.

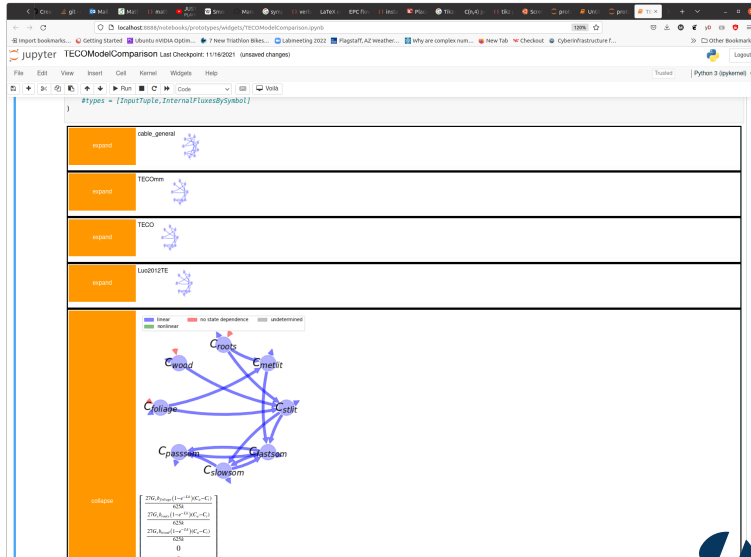
Take home: According to Gödel not all things are computable, but the class of Computable problems is the same regardless of the definition (or language they are expressed in) In particular: **According to C.T it is worth trying to express any result that we can compute somehow as a nested function call.**

REFERENTIAL TRANSPARENCY

“... replacing a subexpression with another one that denotes the same value does not change the value of the expression.” This is the basis for:

- memoization → caching by
 - storing previous argument combinations the function was called with under a unique name
 - remembering the result of a function call by looking up combinations of arguments having occurred before.
- function transformation (derived properties like gradients) , compilation and automatic parallelization

MANY MODELS



Many^{*Many*} = **a lot**

We want to estimate parameters for:

- different models in
- different variants for
- different data sets with
- different sampling algorithms based on
- different ode solvers with
- different parallelization strategies on
- different hardware

and compare the results and the performance.

ADDITIONAL REQUIREMENTS:

- Many computations occur in different comparisons. (caching becomes valuable)
- Some computations can only run on the cluster (caching should work over machine boundaries on file basis)
- Some collaborators have no access to our clusters but still need some of the results
- Some comparisons require query like filtering of results

IMPLEMENTED SOLUTION

- Every job is described as a nested function call (A tree)..
- All leaves are basic data types so that the recursive call can be serialized as a json string (or automatically created python code)
This makes it an extremely versatile (state less) config file.
- The json string is hashable (e.g. sha256 or md5...but **do not use python's hash**)
- The hash can be used to create a unique result directory also for branches of the tree
- If the same task occurs again the hash will be the same, so will the result directory, so that compute intensive parts of the computation can be permanently memoized on disk.

IMPLEMENTED SOLUTION II

- Since the job description can be serialized (written to a json file) it can be executed in different ways.
 - Directly from python,
 - indirectly in parallel subprocesses
 - indirectly in automatically created submission scripts on a cluster
- Although not necessary the json config can also be saved along side the cached results so that the result directory can be traversed and filtered like a database.
- The result directory (including all the hashed results) can be shared between computers (in our case a git subrepo since not all users have cluster access). The sha or md5 hashes are consistent over architectural boundaries the json files are human readable.

SOME GOTCHAS

Not all functions in usual programming languages are functions in the mathematical sense (=pure functions i.e. they do not always yield the same result if given the same arguments)

- obviously random number generators
- less obviously (=nasty) functions whose results are partially randomized e.g. `hash` in python is unique within one interpreter session (the next time you start python your cache id changes...
- All functions that read files are at the mercy of the files not having changed by somebody else on disk... (That is the real reason why pure functional languages like Haskell do not treat them as functions but put them in monadic containers.) → We have to take care ourselves that we do not violate our functional contract.
 - The hash of our recursive function call will not change if we change the source code of one of the constituting functions even if that changes the functionality so that the results would change... In this case we would have to invalidate the cache ...

OTHER BENEFITS OF A FUNCTIONAL PROGRAM DESCRIPTION

- Automatic differentiation
- Automatic compilation for different hardware
- Automatic parallelization (cpus, gpus, tpus)